UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 02

Introducción a C++

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Explicar qué es C++ y en qué contextos se usa.
2. Identificar y acceder a las fuentes de documentación principales.
3. Distinguir entre compiladores e intérpretes, y nombrar los más usados.
4. Compilar y ejecutar un programa simple desde la consola con `g++`.
5. Comprender la estructura mínima de un programa C++ y el rol de cada línea.

> 50 minutos. Sugerido: 15 min expositivo + 20 min demostración en vivo (compilar en consola y abrir el cuaderno) + 15 min práctica guiada.

Secuencia de clase

- **Apertura — ¿Por qué C++? (10 min)**

Abrir con la pregunta: *"¿Qué lenguajes ya conocen?"*. Conectar C++ con aplicaciones que los alumnos usan (Chrome, videojuegos, sistemas embebidos). Mostrar la tabla de características sin entrar en detalle.

> Si el grupo es de primer semestre de ingeniería, enfatizar el acceso a memoria como diferenciador clave frente a Python o Java. Si ya vienen con experiencia, enfocarse en el modelo de compilación y tipado estático.

- **Documentación (5 min)**

Abrir cppreference.com en el navegador y buscar en vivo `std::cout`. Mostrar la estructura de la página: descripción, parámetros, ejemplo, notas. El objetivo es que pierdan el miedo a la documentación técnica desde el primer día.

- **Compiladores y entornos (10 min)**

Explicar la diferencia **compilar vs interpretar** con el diagrama:

$$\texttt{main.cpp} \xrightarrow{\texttt{g++}} \texttt{hola}\ \text{(binario)} \rightarrow \text{ejecuta}$$

Contrasta con cling/xeus-cling: evalúa línea a línea, sin generar binario.

> "Si cling es más cómodo, ¿por qué no se usa en producción?"
> Respuesta esperada: rendimiento, dependencias, no genera binario distribuible.

- **Demostración en consola (15 min)**

  Abrir terminal. Ejecutar en vivo:

```
g++ --version
g++ -std=c++17 main.cpp -o hola
./hola
```

Luego introducir un error a propósito (quitar el punto y coma) y mostrar cómo leer el mensaje de error: `archivo:línea:columna: error: descripción`.

- **Cuaderno Jupyter (10 min)**

  Abrir `sesion02_introduccion_cpp.ipynb`. Ejecutar las celdas secuencialmente. Señalar:

  - No se necesita `main()`.
  - Los `#include` persisten entre celdas.
  - `cin` no funciona interactivamente — mostrar la celda adaptada.

Preguntas frecuentes

"¿Por qué `using namespace std;`? ¿Es mala práctica?"
En proyectos grandes sí: puede causar colisiones de nombres. En código de clase y archivos pequeños es aceptable para reducir ruido visual. Mencionarlo desde el principio forma buenos hábitos.

"¿Qué pasa si omito `return 0;` en `main`?"
Desde C++11 es opcional en `main`. El estándar garantiza retorno 0 implícito. En otras funciones con tipo de retorno distinto de `void`, sí es obligatorio.

"¿`endl` o `n`?"
`n` es más rápido (no hace flush). `endl` asegura que el buffer se vuelca inmediatamente. Para aprendizaje, ambos son equivalentes.

Material de la sesión

- **Cuaderno**: `sesiones/sesion02_introduccion_cpp.ipynb`
- **Ejemplos**: `materialInicial/sesion02/main.cpp`, `main3.cpp`
- **Notas alumno**: `notas_alumno/pdf/sesion02_introduccion_cpp.pdf`

Jueces en línea — Facilitación

Después de la demostración de compilación con `g++`. El juez en línea es la extensión natural: "ya saben compilar localmente; ahora vean cómo funciona cuando lo hace un servidor automáticamente".

• Explicación del concepto (5 min)

Proyecta omegaUp y muestra un problema sencillo (por ejemplo "Hello World" o la suma de dos enteros). Recorre en vivo:

1. El enunciado: entrada, salida, restricciones, ejemplos.
2. El campo de envío: selección de lenguaje, editor o carga de archivo.
3. El veredicto: muestra qué significa cada código (AC, WA, TLE, CE, RE).

Haz un envío en vivo con código correcto (AC) y luego uno incorrecto (WA o CE). Ver el ciclo completo en tiempo real vale más que explicarlo. Si hay acceso a internet limitado, una captura de pantalla del flujo funciona igual.

• omegaUp — Por qué usarlo en clase

- **Mexicano:** fundado en 2011, juez oficial de la OMI. Conectar esto con orgullo local suele motivar más de lo esperado.
- **En español:** los enunciados y la interfaz están en español, eliminando una barrera de entrada.
- **Grupos y concursos:** puedes crear un grupo de clase y asignar problemas directamente. Los alumnos ven su progreso; tú ves quién resolvió qué y en cuántos intentos.
- **Estadísticas:** omegaUp registra intentos, tiempo y distribución de veredictos, útil para identificar alumnos que necesitan apoyo.

Perfil → Cursos → Crear curso. Agrega a los alumnos por su nombre de usuario o comparte el enlace de invitación. Desde el curso puedes asignar problemas con fecha límite y ver el scoreboard en tiempo real durante las sesiones de práctica.

• VJudge — Uso para práctica con problemas internacionales

VJudge permite crear *contests* con problemas de Codeforces, UVa, SPOJ, etc. Es útil cuando quieras usar problemas clásicos que no están en omegaUp. La curva de entrada es un poco mayor; por eso se pide la cuenta ahora pero se usa activamente más adelante en el curso.

- **Tarea: registro de cuentas**

Pide a los alumnos que registren su nombre de usuario de ambas plataformas en un formulario (Google Form, hoja de cálculo compartida o el propio Padlet de la sesión anterior). Necesitarás esos usuarios para agregarlos al grupo de clase en omegaUp.

Rúbrica de práctica

La práctica del cuaderno pide modificar un `cout` con nombre, carrera y año.

Criterio	Puntos	Evidencia
Tres `cout` separados y correctos	4	código ejecutable
Uso correcto de `endl` o `\n`	3	salto de línea visible
Sin errores de compilación	3	celda ejecuta sin error
Total	10	